

Jewellery Web App — GenAI Integration Plan

A phase-by-phase implementation chart for a NestJS + TypeScript + MongoDB application. Each phase introduces only the technologies needed for that stage.

PHASE 1 — LLM FOUNDATION

Goal	Build a simple AI chat endpoint and learn the basic LLM request/response cycle.
Use	• NestJS + TypeScript • LLM provider SDK (OpenAI / Gemini / Claude) • Environment variables • Streaming (optional after basic call)
Flow	User → NestJS → LLM API → Response
Recommended	No LangChain • No RAG • No Vector DB
Focus	Learn: prompts, system/user messages, tokens, structured output, errors.

PHASE 2 — TOOL / FUNCTION CALLING

Goal	Make the LLM interact with your jewellery application's real data and business logic.
Use	• NestJS • LLM SDK tool/function calling • MongoDB / Mongoose • Zod for tool schemas
Flow	User → LLM → selects tool → NestJS executes tool → MongoDB → result → LLM → answer
Recommended	Tools: searchProducts(), getProduct(), getOrderStatus()
Focus	Key concept: the LLM requests a tool; your backend actually executes it.

PHASE 3 — LANGCHAIN.JS

Goal	Replace manually managed AI orchestration with LangChain abstractions once tool calling is understood.
Use	• LangChain.js • LLM provider integration • Tools • Structured output
Flow	NestJS → LangChain Agent → LLM → Tool → MongoDB → LLM → Response
Recommended	Use LangChain for model/tool/agent orchestration.
Focus	Do not add RAG or Vector DB yet. First understand what LangChain is solving.

PHASE 4 — EMBEDDINGS + VECTOR SEARCH

Goal	Add semantic search so users can find jewellery by meaning, not only exact keywords.
Use	• Embedding model (OpenAI / Gemini / other) • MongoDB Atlas Vector Search • MongoDB • LangChain retriever (optional)
Flow	Product/knowledge text → embedding → vector index; user query → embedding → vector search → similar items

Recommended	Use your existing MongoDB ecosystem first; avoid adding Pinecone/Qdrant unless needed.
Focus	Learn: embeddings, chunks, similarity, metadata filters, vector indexes.

PHASE 5 — RAG

Goal	Give the LLM reliable jewellery/business knowledge retrieved from your own documents.
Use	• RAG pattern • MongoDB Atlas Vector Search • Embeddings • LangChain retrievers • LLM
Flow	User question → retrieve relevant chunks → add context → LLM → grounded answer
Recommended	Knowledge: jewellery care, gold/diamond guides, warranty, shipping, returns, FAQs.
Focus	RAG = retrieval + generation. Vector search is the retrieval mechanism.

PHASE 6 — AGENTIC AI

Goal	Combine tools + RAG so the AI can decide which actions/information it needs for a multi-step request.
Use	• LangGraph • LangChain.js • LLM • Tools • MongoDB • Vector Search
Flow	User → Agent → tool/retrieval decision → execute → observe result → next step → final answer
Recommended	Tools: searchProducts(), compareProducts(), getOrderStatus(), checkAvailability(), searchKnowledge()
Focus	Introduce agent workflows only after tools and RAG are individually clear.

PHASE 7 — PRODUCTION AI

Goal	Make the GenAI feature reliable, secure, observable and cost-controlled.
Use	• Authentication/authorization • Rate limiting • Prompt-injection defenses • Logging/tracing • Evaluation • Caching • Redis/BullMQ where needed • LangSmith / OpenTelemetry where useful
Flow	Frontend → NestJS → AI layer → tools/RAG/LLM → monitored response
Recommended	Add fallbacks, timeouts, retries, token/cost controls and test datasets.
Focus	Treat AI as a production subsystem, not just a chatbot endpoint.

Final Technology Stack — After All Phases

Layer	Technology	Why
Frontend	React / Next.js	AI chat/search UI
Backend	NestJS + TypeScript	API, business logic, AI orchestration boundary
LLM	OpenAI / Gemini / Claude	Generation, reasoning, tool selection
AI framework	LangChain.js	Models, tools, retrieval and orchestration
Agent workflow	LangGraph	Multi-step stateful agent workflows
Application DB	MongoDB	Products, users, orders, business data
Vector search	MongoDB Atlas Vector Search	Semantic retrieval
Embeddings	Provider embedding model	Convert text into vectors
RAG	Retrieval + prompt/context pipeline	Ground answers in your data
Optional infra	Redis / BullMQ	Caching and background jobs when required
Observability	LangSmith / OpenTelemetry	Trace and debug AI workflows

Implementation rule: Finish and understand one phase before introducing the next. The first working milestone is Phase 2: an LLM that can call a real `searchProducts()` tool backed by MongoDB.